

Beyond the Answer: Probing Numerical Reasoning in Financial LLMs

Pranav Pothan

LLM Finance RA Coding Test

May 2026

Problem Motivation

Financial QA requires more than reading comprehension.

A system must:

- retrieve numerical values from financial tables,
- interpret surrounding text,
- execute multi-step arithmetic reasoning,
- and generate financially correct answers.

FinQA evaluates:

- symbolic reasoning,
- numerical execution,
- evidence retrieval,
- and financial consistency.

Dataset Overview — FinQA

FinQA contains:

- 8,281 QA pairs
- S&P 500 earnings reports
- tables + text + reasoning programs

Each example contains:

- pre_text
- post_text
- table
- qa.program
- qa.gold_inds
- qa.exe_ans

Example reasoning program:

```
subtract(5829, 5735)  
divide(#0, 5735)
```

Understanding FinQA Programs

What does #0 mean?

Example:

```
subtract(5829, 5735)  
divide(#0, 5735)
```

Execution:

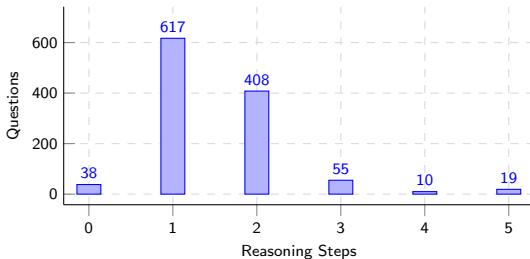
$$5829 - 5735 = 94$$

$$\frac{94}{5735}$$

#0 stores the output of the previous reasoning step.
Programs therefore act as symbolic execution traces.

Common Program Structures in FinQA

Program Length Distribution



Key Observations

- 1-step programs dominate FinQA (53.79%)
- 2-step programs are second most common (35.57%)
- Longer chains (3–5 steps) are rarer but significantly harder

Reasoning Patterns in FinQA Programs

Typical 1-step Operations

```
subtract(a, b)
divide(part, total)
```

Usually correspond to:

- direct arithmetic,
- simple ratios,
- part-to-whole percentages.

Typical 2-step Percentage Reasoning

```
subtract(new, old)
divide(#0, old)
```

Longer Reasoning Chains (3–5 steps)

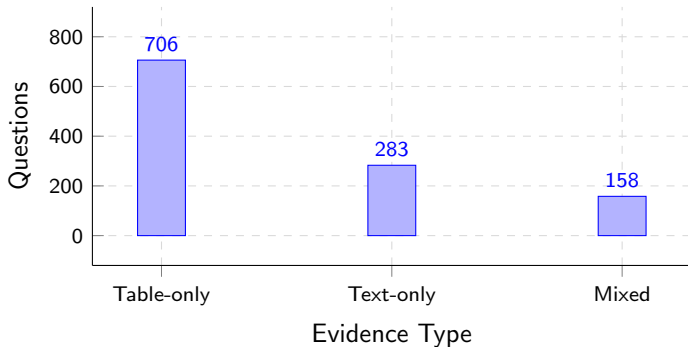
Require:

- intermediate-variable tracking,
- operation-order preservation,
- sequential symbolic execution.

Common in:

- CAGR computation,
- compounded growth analysis,
- shareholder return comparison.

Evidence Source Distribution



Key Observations

- Most FinQA questions are table-centric.
- Mixed-evidence questions require cross-source reasoning.
- Tables dominate numerical grounding in financial QA.

Need for Mixed Retrieval

Failure Using Only Tables

Masco shareholder return example:

- Table contains:

318.46 and 206.49

- Text explains values assume:

\$100 initial investment

- Correct reasoning:

$$\frac{318.46 - 100}{100}$$

- Without text, meaning of:

const_100

is lost.

Failure Using Only Text

Intel leased facilities example:

- Numerical values:

8.1 and 56.0

exist only inside the table

- Required computation:

$$\frac{8.1}{56.0}$$

- Without the table, numerical reasoning becomes impossible.

Therefore, FinQA requires:

- numerical retrieval,
- semantic grounding,
- evidence integration.

Percentage Reasoning Patterns

1. Percentage Growth

```
subtract(new, old)  
divide(#0, old)
```

2. Part-to-Whole Ratio

```
divide(part, total)
```

3. Investment Return

```
subtract(value, const_100)  
divide(#0, const_100)
```

4. Annualized Return

```
subtract(208.14, 100)  
divide(#0, 100)  
divide(const_1, const_5)  
exp(#1, #2)  
subtract(#3, const_1)
```

Uses compound annual growth rate (CAGR):

$$\left(\frac{\text{Final Value}}{\text{Initial Value}} \right)^{\frac{1}{n}} - 1$$

Operation Ordering Failures

1. Percentage Growth Reversal

Correct

```
subtract  
  (153.7,139.9)  
divide(#0,139.9)
```

Wrong

```
subtract  
  (139.9,153.7)  
divide(#0,139.9)
```

2. Wrong Denominator

Correct

```
subtract  
  (4711,4926)  
divide(#0,4926)
```

Wrong

```
divide(#0,4711)
```

3. Investment Return

Correct

```
subtract(121.48,  
  const_100)  
divide(#0,  
  const_100)
```

Wrong

```
divide(121.48,  
  const_100)
```

4. Ratio Reversal

Correct

```
divide(8.1,56.0)
```

Wrong

```
divide(56.0,8.1)
```

Operation Ordering Failures

5. Averaging Before Summation

Correct

```
add(703.1,705.4)
divide(#0,const_2)
```

Wrong

```
divide(703.1,
      const_2)
add(#0,705.4)
```

6. Incorrect Sequential Scaling

Correct

```
multiply
  (766801,40.85)

divide(#0,
      const_1000000
    )
```

Wrong

```
divide(766801,
      const_1000000
    )
multiply(#0,40.85)
```

7. Annualized Return Misordering

Correct

```
subtract
  (208.14,100)
divide(#0,100)
exp(#1,const_0.2)
```

Wrong

```
exp(208.14,const_0
    .2)
subtract(#0,1)
```

Execution vs Program Accuracy

Execution Accuracy: final numerical answer is correct

Program Accuracy: reasoning program exactly matches gold program

A model may produce correct answers even when the symbolic reasoning program differs.

Ex 1 — Percentage Growth

```
Gold:
subtract(153.7,139.9)
divide(#0,139.9)
```

```
Pred:
divide(153.7,139.9)
subtract(#0,1)
```

Both: 9.86%

Ex 2 — Investment Return

```
Gold:
subtract(121.48,const_100)
divide(#0,const_100)
```

```
Pred:
divide(121.48,100)
subtract(#0,1)
```

Both: 21.48%

Ex 3 — Part-to-Whole

```
Gold:
divide(8.1,56.0)
```

```
Pred:
multiply(8.1,100)
divide(#0,56.0)
```

Both: 14.46%

Program accuracy=0 and Execution Accuracy=1

Hypothesis and Experimental Setup

Hypothesis

- 1 retrieval-related failures dominate reasoning errors,
- 2 removing tables degrades performance more than removing text,
- 3 multi-step percentage questions produce highest instability,
- 4 execution accuracy exceeds program accuracy.

Potential Non-Model Failures

- 1 formatting inconsistencies during symbolic execution,
- 2 retrieval ambiguity across similar financial values

Model and Pipeline

Qwen/Qwen2.5-0.5B-Instruct running on a Google Colab T4 GPU.

Pipeline:

- 1 format tables and text,
- 2 construct prompts,
- 3 generate reasoning traces,
- 4 extract answers,
- 5 evaluate execution accuracy,
- 6 store outputs for manual analysis.

Context settings evaluated:

- 1 Full Context
- 2 Table Only
- 3 Text Only

Full-Context Error Taxonomy

Error Type	Frequency
Operation Error	24%
Retrieval Error	18%
Semantic Misunderstanding	17%
Table Grounding Failure	14%
Partial Reasoning Success	9%
Arithmetic Error	8%
Hallucination	6%
Format / Incomplete Output	4%

Strengths over other settings

- balanced access to tables + financial semantics
- fewer hallucinations than text-only
- better contextual understanding than table-only

Weaknesses compared to other settings

- retrieval ambiguity remained high
- distractor numbers caused operand confusion
- long contexts increased reasoning drift

Too much context reduced retrieval stability

Table-Only Error Taxonomy

Error Type	Frequency
Operation Error	29%
Table Grounding Failure	22%
Semantic Misunderstanding	17%
Arithmetic Error	11%
Partial Reasoning Success	8%
Retrieval Error	5%
Hallucination	4%
Format / Incomplete Output	4%

Strengths over other settings

- lowest retrieval error rate
- fewer distractor-number confusions
- more stable numerical grounding

Weaknesses compared to other settings

- removed financial semantic explanations
- denominator selection remained unstable
- symbolic reasoning errors became exposed

Grounding improved, reasoning still weak

Text-Only Error Taxonomy

Error Type	Frequency
Retrieval Error	32%
Semantic Misunderstanding	18%
Operation Error	16%
Hallucination	12%
Format / Incomplete Output	9%
Partial Reasoning Success	5%
Arithmetic Error	5%
Table Grounding Failure	3%

Strengths over other settings

- preserved financial narrative context
- retained semantic descriptions of metrics

Weaknesses compared to other settings

- highest retrieval instability
- severe hallucination increase
- missing numerical grounding caused failures

Tables were critical for financial reasoning

Program vs Answer Evaluation

Execution Accuracy

Correct final numerical answer

Program Accuracy

Correct symbolic reasoning program

Setting	Exec.	Prog.
Full Context	1%	1%
Table-Only	1%	1%
Text-Only	0%	0%

Exec. Acc. > Prog. Acc.

Full-Context Example

```
Gold:
divide(817388,3644331)

Model:
total_debt / total_assets

Pred: 0.225
Gold: 0.22429
```

Exec = 1 Prog = 1

Table-Only Example

```
Gold:
subtract(5829,5735)

Model:
operation(net_revenue,2015)

Pred: 94
Gold: 94.0
```

Exec = 1 Prog = 1

Layer 3 — Evidence-Grounded Retrieval

Motivation from Layer 2

Most failures originated from:

- retrieval instability,
- distractor-number confusion,
- wrong operand selection,
- semantic grounding failures.

Key insight:

Many failures occurred before arithmetic execution

Core Idea

Instead of using:

- full financial tables,
- surrounding text,
- all numerical values,

the model receives only:

```
qa.model_input
```

containing:

- retrieved evidence rows,
- relevant financial statements,
- question-specific context.

Goal:

Reduce retrieval ambiguity before reasoning begins

Why This Helps

- reduces context entropy,
- reduces distractor interference,
- improves operand alignment,
- improves fiscal-year selection.

Evidence-Grounded Retrieval Results

Example 1 — Retrieval Improvement

Evidence:
2014 net revenue = 5735
2015 net revenue = 5829

Reasoning:
 $5829 - 5735 = 94$

Predicted:

94

Gold:

94.0

Correct operand retrieval improved reasoning stability.

Example 2 — Better Fiscal-Year Alignment

Question:
Increase/decrease in 2012 over 2011?

Evidence:
2012 = 65
2011 = 53

Reasoning:
 $65 - 53 = 12$

Predicted:

12

Gold:

12.0

Conclusion and Future Work

Key Takeaways

- Most failures came from retrieval and grounding errors
- Tables were critical for stable financial reasoning
- Execution accuracy was often higher than program accuracy
- Evidence grounding reduced distractor confusion
- Context quality mattered more than model size

Future Work

With more GPU resources, I would:

- Build a better evidence reranker,
- Add symbolic program verification,
- Test self-consistency reasoning.